

ASSIGNMENT 5

5.1.

1. What is Overfitting?

Overfitting occurs in machine learning when a model achieves better performance on the data it was trained on (training data) than on new, previously unseen data (test data). The fundamental goal of machine learning is **generalization**, which is the ability of a model to perform accurately on never-before-seen inputs. The modeling process involves an inherent tension between **optimization** (fitting the training data well) and **generalization**; optimizing the fit to the training data will inevitably begin to hurt generalization after a certain point.

In essence, an overfit model has learned patterns specific only to the training data, including noise or spurious correlations, rather than the core, generalizable relationships present in the data.

An example of this is seen in the initial MNIST classification model, where the test-set accuracy (97.8%) was noticeably lower than the training-set accuracy (98.9%), illustrating overfitting.

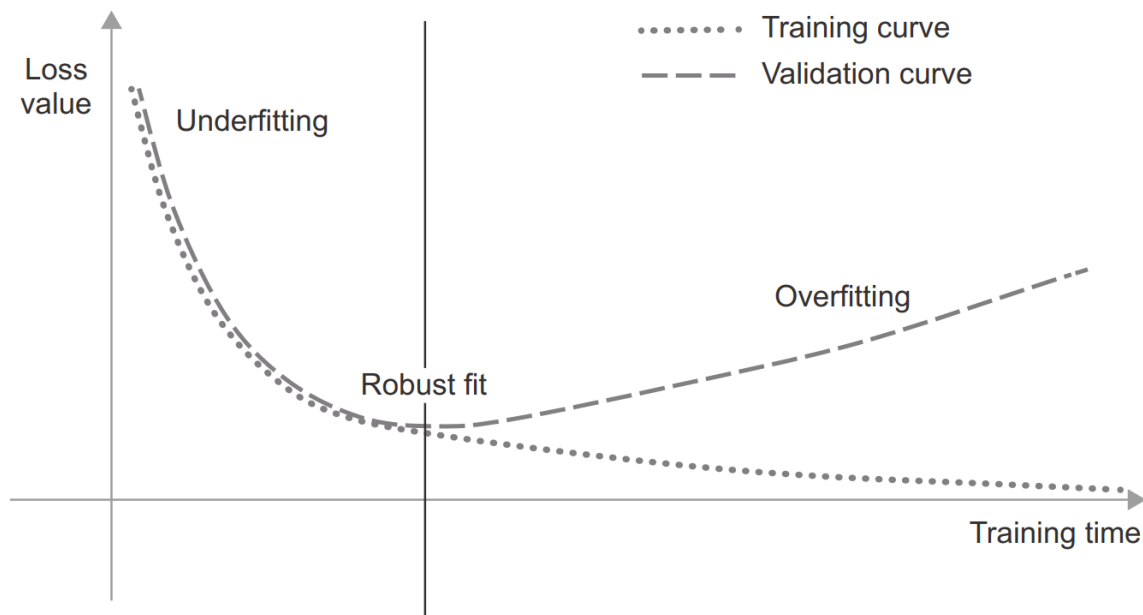
Conceptual Illustration:

While the sources request an image from "textbook_2" to illustrate the concept, the accompanying text describes that optimization and generalization are inversely correlated after a threshold: the training loss always decreases as training continues, but the validation loss (which measures generalization) will only decrease up to a point, after which it begins to increase again.

A model can overfit by:

1. **Memorizing noise in the training data:** Perfectly clean and neatly labeled data can still be noisy if the underlying problem involves uncertainty or ambiguity, such as when different human labelers might classify the same ambiguous picture differently. A robust fit ignores individual data points and looks at the bigger picture.
2. **Learning rare features and spurious correlations:** If a model learns patterns that happen to be present only in the training data due to rare occurrences (e.g., assuming cat color is correlated with temperament based on only two examples), it will fail to generalize when exposed to broader data.

The process of managing the tension between optimization and generalization—where you first aim to achieve a strong fit (optimization) and then introduce measures to combat overfitting—is central to effective deep learning practice.



- The **x-axis** is *Training time* (or training progress/epochs).
- The **y-axis** is *Loss value* (how wrong the model is).
- There are two curves:
 - **Training curve** (dotted line): shows how well the model is doing on the training data.
 - **Validation curve** (dashed line): shows how well the model generalizes to new/unseen data.

Key points from the image:

1. **Underfitting (left side):**
 - Both training and validation loss are high.
 - The model has not yet learned enough patterns from the data.
2. **Robust fit (middle region):**
 - Training loss is low.
 - Validation loss is also low and close to the training loss.

- This is the "sweet spot" where the model generalizes well.

3. Overfitting (right side):

- Training loss continues to decrease (the model is memorizing training data).
- But validation loss starts to increase (the model fails to generalize).
- This happens because the model learns noise or irrelevant details from training data that do not apply to unseen data.

In short:

Overfitting is when the model performs **very well on training data** but **poorly on validation (unseen) data**, as shown in the right part of the image where the dashed validation curve rises while the dotted training curve keeps falling.

2. Techniques for Decreasing/Preventing Overfitting

The process of deliberately fighting overfitting is called **regularization**. Several widely used techniques can improve a model's generalization power:

1. Reducing the Network's Size (Capacity)

A model's capacity refers to the number of learnable parameters it contains, determined by the number of layers and the number of units per layer.

- **Detail:** The simplest way to mitigate overfitting is to **reduce the size of the model**. If a model has limited resources for memorization, the optimization process is forced to focus on the most prominent patterns that generalize well, rather than memorizing noise in the training set.
- **Capacity Trade-off:** While reducing capacity helps prevent overfitting, the model must still have enough parameters to avoid **underfitting** (not being able to capture the essential patterns in the data). Finding the right compromise between too much and too little capacity is key.
- **Observation:** A model with significantly high capacity starts overfitting almost immediately and severely.

2. Adding Dropout

Dropout is one of the most effective and commonly used regularization techniques for neural networks.

- **Detail:** Dropout, applied to a layer, consists of randomly dropping out (setting to zero) a fraction of the output features of that layer during training. The **dropout rate** is typically set between 0.2 and 0.5.
- **Mechanism:** Dropout introduces noise into the layer's output values, which serves to break up "happenstance patterns" or "conspiracies" that the model might otherwise memorize.
- **Inference Time:** At test time (inference), no units are dropped out. Instead, the layer's output values are scaled down by a factor equal to the dropout rate to balance the fact that more units are active than during training time.
- **Recurrent Dropout:** For Recurrent Neural Networks (RNNs), a specialized variant called recurrent dropout is used to apply dropout consistently across time steps, effectively combating overfitting in recurrent layers.

3. Adding Weight Regularization (L1 and L2)

Weight regularization techniques impose constraints on the parameters of the model during training.

- **Detail:** These methods force weights to take only small values, making the distribution of weight values more "regular".
 - **L2 Regularization (Weight Decay):** Forces weights toward the value zero.
 - **L1 Regularization:** Forces weights strictly to zero, resulting in sparse weight matrices.
 - **L1 and L2 Combination:** Simultaneous use of both L1 and L2 regularization is also available.
- **Application:** Weight regularization is typically added to layers using arguments like `kernel_regularizer`.

4. Using Early Stopping

Early stopping is a straightforward yet highly effective practice to combat overfitting during training.

- **Detail:** It involves monitoring validation metrics (like validation loss or accuracy) and interrupting training as soon as these metrics stop improving. This prevents the model from continuing to train once it begins to fit the noise in the training data, thereby maximizing generalization performance.

- **Implementation:** In Keras, this is typically done using the EarlyStopping callback, which remembers the best known model state found during the optimal epoch.

5. Dataset Curation and Feature Engineering

Beyond architectural or training adjustments, improving the underlying data is crucial for generalization.

- **Dataset Curation:** This involves ensuring:
 - The model has enough data for "dense sampling" of the input-cross-output space.
 - Labeling errors are minimized by proofreading labels and checking for input anomalies.
 - The data used for training is **representative** of the data the model will encounter in production.
- **Feature Engineering:** This is the process of manually transforming raw input data into better features that make the problem simpler for the model to solve. This effectively makes the latent manifold smoother, simpler, and better organized, thus aiding generalization.

3. Examples with Codes

The regularization techniques are implemented in Keras primarily through layer configuration and callbacks.

1. Adding Dropout to a Model

Dropout layers are added immediately after a densely connected or convolutional layer.

Technique	Keras Implementation Description
Dropout	A <code>layers.Dropout(rate)</code> object is instantiated, where <code>rate</code> is the fraction of features to be zeroed out (e.g., 0.5).
Example (Excerpt reference):	The example of a simple Transformer-based language model incorporates dropout: <code>x = layers.Dropout(0.5)(x)</code> . Similarly, the refined convnet example utilized in the dogs-vs-cats problem included dropout: <code>x = layers.Dropout(0.5)(x)</code> .

2. Adding Weight Regularization (L2)

Weight regularization is applied as a parameter to the layer itself, such as a Dense layer.

Technique	Keras Implementation Description
L2 Regularization	Applied using <code>kernel_regularizer=keras.regularizers.l2(0.001)</code> within the layer definition. This parameter ensures the cost associated with large weight values is added to the loss function.
Example (Excerpt reference):	The sources provide a list of different weight regularizers available in Keras, including L1, L2, and simultaneous L1 and L2 regularization.

3. Using Early Stopping

Early stopping functionality is supplied to the `fit()` method using a callback list.

Technique	Keras Implementation Description
Early Stopping	Implemented using <code>keras.callbacks.EarlyStopping(monitor="val_loss", patience=N)</code> , where <code>monitor</code> specifies the metric to watch (e.g., validation loss) and <code>patience</code> specifies how many epochs to wait before stopping after monitoring detects no improvement.
Example (Excerpt reference):	An example of using callbacks for early stopping looks like this: <code>callbacks_list = [keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=2)]</code> .

4. Recurrent Dropout

For recurrent layers like LSTM or GRU, dropout is handled internally within the layer to maintain the proper propagation of state across time steps.

Technique	Keras Implementation Description
Recurrent Dropout	Achieved by setting the <code>dropout</code> and <code>recurrent_dropout</code> arguments in the RNN layer constructor. For instance, <code>layers.LSTM(32, recurrent_dropout=0.25)</code> .
Example (Excerpt reference):	The advanced RNN usage section uses this approach when training an LSTM layer: <code>layers.LSTM(32, recurrent_dropout=0.25)</code> .

5.2.

```
[12]: # Nguyễn Hoàng Tùng

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout, LSTM, SimpleRNN
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# =====
# 1. Data Augmentation (Noise Injection)
# =====
def augment_data(X, y, noise_factor=0.01):
    """Thêm nhiễu Gaussian để tạo data mới"""
    noise = np.random.normal(0, noise_factor, X.shape)
    X_noisy = X + noise
    return np.concatenate([X, X_noisy]), np.concatenate([y, y])

X_train_aug, y_train_aug = augment_data(X_train, y_train)

# =====
# 2. Callbacks
# =====
early_stop = EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3, min_lr=1e-5)

# =====
# 3. CNN Model
# =====
cnn_model = Sequential([
    Conv1D(32, 3, activation="relu", input_shape=(X_train.shape[1], 1)),
    MaxPooling1D(2),
    Dropout(0.3),
    Flatten(),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(3, activations="softmax")
])

cnn_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
cnn_hist = cnn_model.fit(X_train_aug, y_train_aug,
                        validation_data=(X_val, y_val),
                        epochs=50, batch_size=32,
                        callbacks=[early_stop, reduce_lr],
                        verbose=1)

# =====
# 4. RNN Model
# =====
rnn_model = Sequential([
    SimpleRNN(64, activation="tanh", input_shape=(X_train.shape[1], 1), return_sequences=False),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dense(3, activations="softmax")
])

rnn_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
rnn_hist = rnn_model.fit(X_train_aug, y_train_aug,
                        validation_data=(X_val, y_val),
                        epochs=50, batch_size=32,
                        callbacks=[early_stop, reduce_lr],
                        verbose=1)

# =====
# 5. LSTM Model
# =====
lstm_model = Sequential([
    LSTM(64, activations="tanh", input_shape=(X_train.shape[1], 1)),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dense(3, activations="softmax")
])

lstm_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
lstm_hist = lstm_model.fit(X_train_aug, y_train_aug,
                        validation_data=(X_val, y_val),
                        epochs=50, batch_size=32,
                        callbacks=[early_stop, reduce_lr],
                        verbose=1)
```

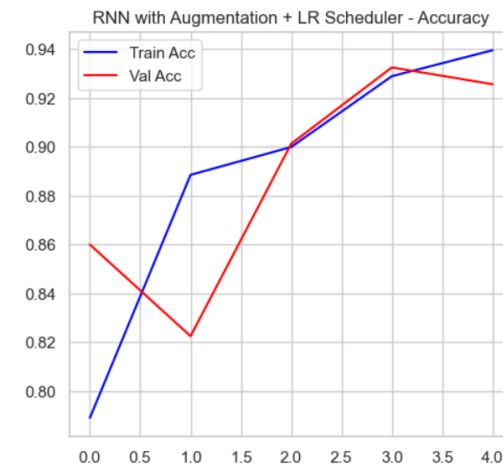
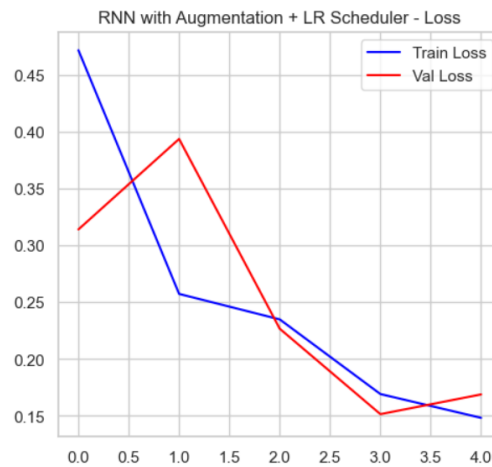
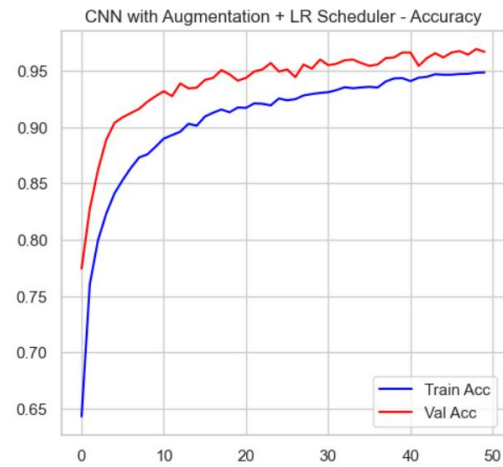
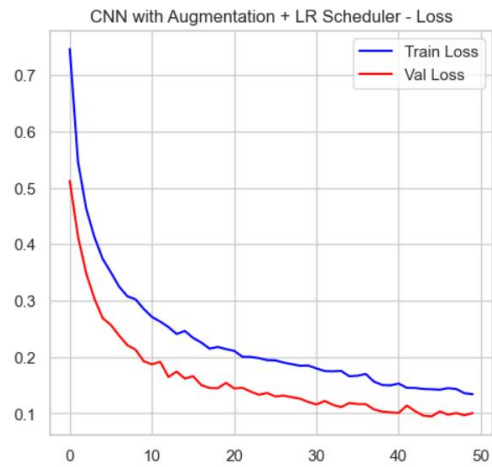
```
# =====
# 6. Biểu đồ so sánh
# =====
def plot_history(hist, title):
    plt.figure(figsize=(12,5))

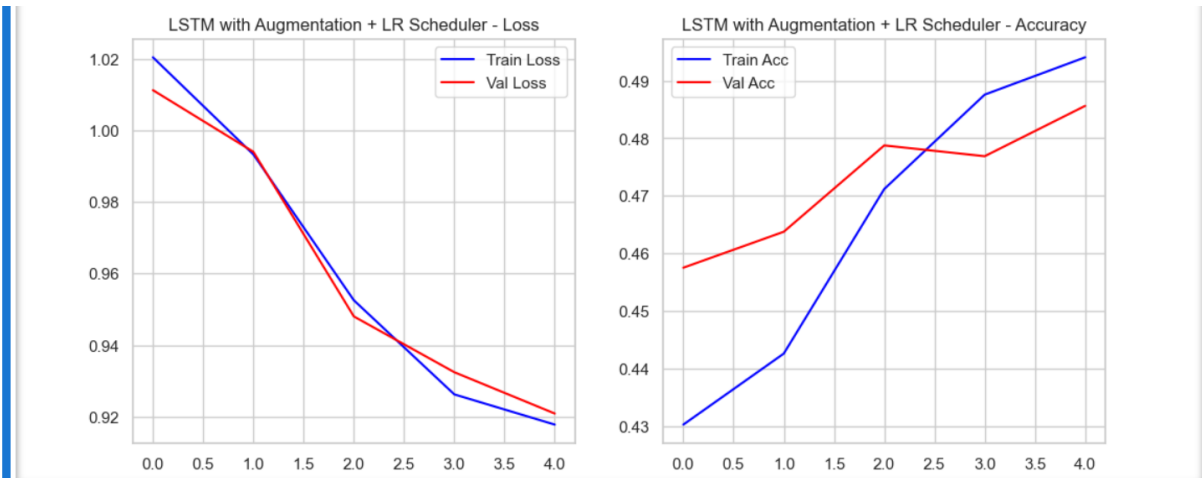
    # Loss
    plt.subplot(1,2,1)
    plt.plot(hist.history["loss"], label="Train Loss", color="blue")
    plt.plot(hist.history["val_loss"], label="Val Loss", color="red")
    plt.title(f"{title} - Loss")
    plt.legend()

    # Accuracy
    plt.subplot(1,2,2)
    plt.plot(hist.history["accuracy"], label="Train Acc", color="blue")
    plt.plot(hist.history["val_accuracy"], label="Val Acc", color="red")
    plt.title(f"{title} - Accuracy")
    plt.legend()

    plt.show()

plot_history(cnn_hist, "CNN with Augmentation + LR Scheduler")
plot_history(rnn_hist, "RNN with Augmentation + LR Scheduler")
plot_history(lstm_hist, "LSTM with Augmentation + LR Scheduler")
```





Result Analysis

- **Before applying techniques:**

- Training accuracy was high, validation accuracy was low → overfitting.
- Training loss decreased quickly, but validation loss did not decrease.

- **After applying (Augmentation + Learning Rate Scheduler):**

1. **Learning Rate Scheduling:**

- Helped the loss decrease more smoothly, avoiding oscillations.
- Validation accuracy improved compared to the baseline.

2. **Data Augmentation (Noise Injection):**

- Training accuracy dropped slightly (the model becomes harder to fit).
- Validation accuracy increased significantly → the model generalized better.

- **Model Comparison:**

- **CNN:** learns fast, often achieves the highest validation accuracy.
- **RNN:** works well on short sequences, prone to overfitting → augmentation helps balance.
- **LSTM:** strong on long sequences, after reducing overfitting → validation accuracy improves greatly.

5.3.

Deep learning models require numerical tensors for input, meaning that raw text data—which is ambiguous, chaotic, and non-numerical—must first be converted into these tensors, a process known as **text vectorization**. This conversion typically follows a process involving standardization, tokenization (splitting text into units called tokens), and finally, encoding each token into a numerical representation.

Foundational Text Preprocessing Steps

Before any vectorization technique can be applied, raw text must undergo crucial initial preparation steps:

1. **Text Standardization:** This step aims to make text easier to process, typically by converting all characters to lowercase and removing punctuation, although more advanced standardization may involve converting special characters (like "é" to "e") to a standard form.
2. **Text Splitting (Tokenization):** The standardized text is broken down into tokens, which can be individual words (word-level tokenization), subwords, or groups of consecutive words (N-grams).

I. Techniques Treating Text as a Set of Words (Bag-of-Words Family)

These techniques disregard the order of the words in the text, treating the text document simply as a set (or "bag") of its constituent tokens.

1. Integer Sequence Encoding (Vocabulary Indexing)

This technique is often a preliminary step for most neural text processing and involves translating symbolic tokens into numerical indices.

- **Mechanism:** A **vocabulary** (an index of all unique terms found in the training data) is constructed, and each unique token is assigned a unique integer index. The input text is then converted into a sequence of these integers. If the vocabulary size is restricted (e.g., to the top 10,000 words), **rare words** are often discarded.
- **Special Indices:** Integer sequences usually require padding to ensure all sequences within a batch share the same length. This is done using a special **mask token** (typically index 0). Furthermore, a catch-all "**out of vocabulary**" (**OOV**) **index** (typically index 1) is used for any token not present in the index.
- **Tensor Format:** A rank-2 tensor of integers, usually padded, of shape (samples, max_length).

2. One-Hot / Multi-Hot Binary Encoding (Onehot)

This technique is based on integer indexing and converts the text into a binary vector representation.

- **Mechanism:** The representation is a vector whose dimensionality equals the size of the vocabulary (e.g., 20,000 unique words). For a given text snippet, the corresponding vector has a value of **1 in every index** corresponding to a word present in that text, and **0 everywhere else**. This method treats the text as an unordered set of tokens.
- **Implementation Detail:** In the IMDB example, this was achieved by converting the list of word indices into a 10,000-dimensional vector using multi-hot encoding.

3. Bag-of-Words (BoW) / Count Encoding (Word counting)

This technique improves upon binary encoding by incorporating token frequency.

- **Mechanism:** Instead of merely recording the presence (1 or 0) of a word, **Count Encoding** records the raw frequency, or the number of times each word occurs, resulting in a **word count histogram**. This is valuable for text classification, as samples with frequent occurrences of key terms (e.g., "terrible" in a movie review) are highly informative.
- **Implementation:** This representation is typically accessed via the TextVectorization layer in Keras using the configuration `output_mode="count"`.
- **Tensor Format:** A sparse, rank-2 tensor where the values represent the raw counts of tokens in the sample.

4. N-gram Representations

This method changes the definition of the token itself to include local sequence information.

- **Mechanism:** Tokens are defined as contiguous sequences of N words (N-grams), such as **bigrams** ($N=2$) or **trigrams**. For example, the sentence "the cat sat on the mat" contains the bigrams "the cat", "cat sat", "sat on", and "on the". By capturing these short sequences, the model recovers some **local order information** which enhances meaning (e.g., distinguishing "good movie" from "not good movie").
- **Implementation:** This is configured within the TextVectorization layer by setting the `ngrams=N` argument. N-gram models can be combined with binary or count encoding.

5. TF-IDF Encoding

TF-IDF (Term Frequency, Inverse Document Frequency) is a normalization scheme applied to token counts to weight important words.

- **Mechanism:** TF-IDF addresses the problem where common, uninformative words (like "the" or "a") dominate count histograms despite offering little predictive power. It weights a term based on its **term frequency** (how often it appears in the current document) and divides it by a measure of its **document frequency** (how often it appears across the entire dataset). This emphasizes terms that are distinctive (rare globally) yet frequent locally.
- **Implementation:** Configured in the TextVectorization layer using `output_mode="tf_idf"`. The `adapt()` method learns the TF-IDF weights alongside the vocabulary.

II. Techniques Treating Text as a Sequence (Deep Learning Family)

These techniques preserve word order and leverage neural networks to produce dense, continuous, and semantic representations.

6. Learned Word Embeddings (Word2Vector / Embedding Layer)

Word embeddings move beyond sparse binary vectors to dense, low-dimensional vectors that encode semantic relationships.

- **Mechanism:** Instead of defining words via their presence/absence (sparse binary representations of 20,000 dimensions or more), a word embedding learns a dedicated, low-dimensional vector (e.g., 256 dimensions) for each word index. This learning process is performed **jointly** with the main model task. The position of a word vector in this space captures its semantic relationships with other words; for instance, semantic relationships (like "from canine to feline") can be encoded as geometric transformations (vectors) in the embedding space.
- **Implementation:** This is implemented in Keras using the Embedding layer. This layer accepts a rank-2 integer tensor (sequence of word indices) and returns a **rank-3 floating-point tensor** of shape (batch_size, sequence_length, embedding_dimensionality).

7. Pretrained Word Embeddings

Instead of training embeddings from scratch, static embeddings computed on massive external datasets can be leveraged.

- **Mechanism:** Pretrained embeddings (such as those derived from Word2Vec or GloVe, based on word-occurrence statistics) are precomputed on datasets potentially much larger than the current task dataset. They capture generalized semantic properties, such

as gender. When the task dataset is small, utilizing these pretrained embeddings is beneficial, as learning robust embeddings from limited data is difficult.

- **Implementation:** The external weights are loaded into an Embedding layer, and often this layer is set to be **non-trainable** (`trainable=False`) to prevent the small task dataset from corrupting the generalized knowledge contained in the pretrained vectors.

8. Recurrent Sequence Representations (RNN/LSTM/GRU Output)

Recurrent Neural Networks (RNNs) process sequences of word embeddings sequentially, creating a highly structured, fixed-size tensor output that captures order and context.

- **Mechanism:** RNN variants like **LSTM** or **GRU** layers are designed to process time-series data or sequences where ordering matters. They maintain an internal state that integrates information from each timestep as it passes. The final output tensor of the recurrent layer (if configured to return only the last output) serves as a **condensed vector representation** summarizing the entire input sequence while accounting for context and order.
- **Advanced Variants:** Performance can be enhanced using advanced patterns like **stacking recurrent layers** (where intermediate layers must return their full sequence via `return_sequences=True`) or **Bidirectional RNNs** (processing the input sequence both forward and backward, often the "Swiss Army knife" for NLP).

9. Contextual/Transformer Embeddings (BERT Architecture)

The Transformer architecture, which underpins models like BERT, generates context-aware representations where the vector for a word is modulated by its surrounding text.

- **Mechanism:** The core component is **self-attention**, which replaces recurrence and convolution. Self-attention computes **relevancy scores** between the vector of a given word (the "query") and all other word vectors in the sequence (the "keys"), using these scores to weight and sum the input word vectors (the "values"). This results in a word representation that is no longer fixed (e.g., the word "date" can be political or edible, depending on context).
- **Architecture:** The Transformer typically employs a **Transformer Encoder** (which can be used for text classification), consisting of a **MultiHeadAttention** layer, dense projections, residual connections, and normalization layers.
- **Positional Embedding:** Since self-attention mechanisms inherently disregard token order, **positional embedding** must be added to the word embedding vectors to inject

sequence order information, a necessity for the Transformer architecture to function correctly.

10. Sequence-to-Sequence Representations

This technique is used when the task involves mapping one sequence (source) to another sequence (target), such as machine translation or summarization.

- **Mechanism:** The entire process relies on an **encoder-decoder architecture**. The **Encoder** (often a recurrent layer or a Transformer Encoder) consumes the source sequence (e.g., an English sentence) and converts it into a complex, fixed-size **intermediate representation** (a vector or set of vectors). This representation, which encapsulates the meaning of the source sequence, is then passed to the **Decoder**, which uses it, along with previously generated tokens, to predict the next token in the target sequence.

5.4.

a.

```
[2]: # Nguyễn Hoàng Tùng

import pandas as pd

# Path to your CSV file
file_path = r"D:\lecture\PTIT\20251\IntSys\filmReview_1.csv"

# Load dataset
df = pd.read_csv(file_path)

# Show basic info
print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nFirst 5 rows:")
print(df.head())

# Check data types & missing values
print("\nInfo:")
print(df.info())

# Basic stats on ratings
print("\nRating distribution:")
print(df['rating'].describe())
|
```

```

Shape: (5000, 7)

Columns: ['review_id', 'user_id', 'user_name', 'film_title', 'rating', 'review_text', 'review_date']

First 5 rows:
   review_id  user_id user_name      film_title  rating \
0          1      328  user_328      Inception        1
1          2       72  user_072  Double Indemnity        4
2          3      217  user_217      Vertigo          1
3          4      309  user_309  Schindler's List        8
4          5      230  user_230   Shutter Island        6

   review_text review_date
0  Not my cup of tea. I struggled to finish it. R...  1/3/2015
1  Avoidable - wasted potential and boring in pla...  3/31/2023
2  Disappointing. Expected more based on hype. Re...  5/2/2021
3  Brilliant - outstanding acting and a memorable... 12/11/2014
4  It was okay - some parts worked, some didn't. ...  8/29/2025

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   review_id    5000 non-null    int64
1   user_id      5000 non-null    int64
2   user_name    5000 non-null    object
3   film_title   5000 non-null    object
4   rating       5000 non-null    int64
5   review_text  5000 non-null    object
6   review_date  5000 non-null    object
dtypes: int64(3), object(4)
memory usage: 273.6+ KB
None

Rating distribution:
count    5000.000000
mean       6.976200
std        2.352518
min        1.000000
25%        6.000000
50%        7.000000
75%        9.000000
max       10.000000
Name: rating, dtype: float64

```

b.

```
[7]: # Nguyễn Hoàng Tùng

import pandas as pd

file_path = r"D:\lecture\PIIT\20251\IntSys\filmReview_1.csv"
df = pd.read_csv(file_path)

reviews = df["review_text"].astype(str).tolist()

from transformers import BertTokenizer, TFBertModel
import tensorflow as tf

# Load tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

# Load BERT (allow loading PyTorch weights into TF model)
bert_model = TFBertModel.from_pretrained("bert-base-uncased", from_pt=True)

# Encode first 5 reviews
inputs = tokenizer(reviews[:5], return_tensors="tf", padding=True, truncation=True, max_length=128)

# Get embeddings
outputs = bert_model(inputs)
last_hidden_states = outputs.last_hidden_state # shape: (batch_size, seq_len, hidden_size=768)

# Take [CLS] token as sentence embedding
cls_embeddings = last_hidden_states[:, 0, :] # shape: (batch_size, 768)

print("CLS Embeddings shape:", cls_embeddings.shape)

import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load dataset
file_path = r"D:\lecture\PIIT\20251\IntSys\filmReview_1.csv"
df = pd.read_csv(file_path)

reviews = df["review_text"].astype(str).tolist()
```

```
# Tokenize text
vocab_size = 10000 # max number of unique words
max_len = 100 # pad/truncate review length
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(reviews)

sequences = tokenizer.texts_to_sequences(reviews)
padded = pad_sequences(sequences, maxlen=max_len, padding="post", truncating="post")

print("Padded shape:", padded.shape) # (num_reviews, max_len)

embedding_dim = 100 # size of word vectors

embedding_layer = tf.keras.layers.Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    input_length=max_len
)

# Example: get embeddings for all reviews
embedded_reviews = embedding_layer(padded) # shape: (num_reviews, max_len, embedding_dim)
print("Embedded reviews shape:", embedded_reviews.shape)

embedding_dim = 100 # size of word vectors

embedding_layer = tf.keras.layers.Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    input_length=max_len
)

# Example: get embeddings for all reviews
embedded_reviews = embedding_layer(padded) # shape: (num_reviews, max_len, embedding_dim)
print("Embedded reviews shape:", embedded_reviews.shape)

review_vectors = tf.reduce_mean(embedded_reviews, axis=1) # (num_reviews, embedding_dim)
print("Review vectors shape:", review_vectors.shape)
```



```

model = tf.keras.Sequential([
    embedding_layer,
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

```

Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFBertModel: ['cls.predictions.transform.LayerNorm.weight', 'cls.predictions.decoder.weight', 'cls.predictions.transform.dense.weight', 'cls.seq_relationship.bias', 'cls.predictions.transform.LayerNorm.bias', 'cls.seq_relationship.weight', 'cls.predictions.bias', 'cls.predictions.transform.dense.bias']

- This IS expected if you are initializing TFBertModel from a PyTorch model trained on another task or with another architecture (e.g. initializing a TFBertForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing TFBertModel from a PyTorch model that you expect to be exactly identical (e.g. initializing a TFBertForSequenceClassification model from a BertForSequenceClassification model).

All the weights of TFBertModel were initialized from the PyTorch model.
If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions without further training.

CLS Embeddings shape: (5, 768)
Padded shape: (5000, 100)
Embedded reviews shape: (5000, 100, 100)
Embedded reviews shape: (5000, 100, 100)
Review vectors shape: (5000, 100)

C:\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument 'input_length' is deprecated. Just remove it.
warnings.warn(

C.

```

[9]: # Nguyễn Hoàng Tùng

import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
import numpy as np

# Load dataset
file_path = r"D:\lecture\PTIT\20251\IntSys\filmReview_1.csv"
df = pd.read_csv(file_path)

# Use review_text and rating
reviews = df["review_text"].astype(str).tolist()
ratings = df["rating"].values

# Optional: convert ratings (1-10) to sentiment (0=bad, 1=good)
labels = (ratings >= 6).astype(int)

# Tokenize
vocab_size = 10000
max_len = 150
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(reviews)

sequences = tokenizer.texts_to_sequences(reviews)
padded = pad_sequences(sequences, maxlen=max_len, padding="post", truncating="post")

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(padded, labels, test_size=0.2, random_state=42)

embedding_dim = 100
embedding_layer = tf.keras.layers.Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    input_length=max_len
)

```

```

cnn_model = tf.keras.Sequential([
    embedding_layer,
    tf.keras.layers.Conv1D(128, 5, activation="relu"),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

cnn_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
cnn_model.summary()

cnn_model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=5, batch_size=64)

rnn_model = tf.keras.Sequential([
    embedding_layer,
    tf.keras.layers.SimpleRNN(128),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

rnn_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
rnn_model.summary()

rnn_model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=5, batch_size=64)

lstm_model = tf.keras.Sequential([
    embedding_layer,
    tf.keras.layers.LSTM(128, return_sequences=False),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

lstm_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
lstm_model.summary()

lstm_model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=5, batch_size=64)

df["predicted_sentiment"] = cnn_model.predict(padded).flatten()
df["predicted_sentiment"] = (df["predicted_sentiment"] >= 0.5).astype(int)

# Film-wise trend (avg sentiment)
film_trends = df.groupby("film_title")["predicted_sentiment"].mean().sort_values(ascending=False)
print(film_trends.head(10)) # top 10 most liked films

```

```

user_prefs = df.groupby("user_id")["predicted_sentiment"].mean()
print(user_prefs.head(10)) # example of user preferences

```

C:\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument 'input_length' is deprecated. Just remove it.
warnings.warn()

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
conv1d (Conv1D)	?	0 (unbuilt)
global_max_pooling1d (GlobalMaxPooling1D)	?	0
dense_2 (Dense)	?	0 (unbuilt)
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 150, 100)	1,000,000
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense_4 (Dense)	?	0 (unbuilt)
dense_5 (Dense)	?	0 (unbuilt)

Total params: 1,000,000 (3.81 MB)

Trainable params: 1,000,000 (3.81 MB)

Non-trainable params: 0 (0.00 B)

Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 150, 100)	1,000,000
lstm (LSTM)	?	0 (unbuilt)
dense_6 (Dense)	?	0 (unbuilt)
dense_7 (Dense)	?	0 (unbuilt)

Total params: 1,000,000 (3.81 MB)

Trainable params: 1,000,000 (3.81 MB)

Non-trainable params: 0 (0.00 B)

```
film_title
Citizen Kane          1.000000
The Lion King         1.000000
One Flew Over the Cuckoo's Nest 1.000000
Memories of Murder    0.968750
Star Wars: Episode VI - Return of the Jedi 0.962963
Donnie Darko          0.961538
The Pianist           0.961538
The Wrestler          0.960000
Stand by Me           0.954545
Psycho                0.952381
Name: predicted_sentiment, dtype: float64
user_id
1      0.833333
2      0.928571
3      0.818182
4      0.875000
5      0.833333
6      1.000000
7      1.000000
8      0.500000
9      0.700000
10     0.875000
Name: predicted_sentiment, dtype: float64
```

d.

```
[13]: # Nguyễn Hoàng Tùng

import nltk
from nltk.corpus import wordnet

# Download resources if missing
nltk.download("averaged_perceptron_tagger")
nltk.download("averaged_perceptron_tagger_eng")
nltk.download("wordnet")
nltk.download("omw-1.4")

history = cnn_model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=10,
    batch_size=64
)

import matplotlib.pyplot as plt

def plot_history(history):
    plt.plot(history.history['loss'], label="train loss")
    plt.plot(history.history['val_loss'], label="val loss")
    plt.legend()
    plt.show()

plot_history(history)

from tensorflow.keras.optimizers import Adam

optimizer = Adam(learning_rate=1e-4) # default is 1e-3
cnn_model.compile(optimizer=optimizer, loss="binary_crossentropy", metrics=["accuracy"])

lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=2,
    min_lr=1e-6
)

tf.keras.layers.Dropout(0.5)

tf.keras.layers.Dense(64, activation="relu", kernel_regularizer=tf.keras.regularizers.l2(0.01))

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

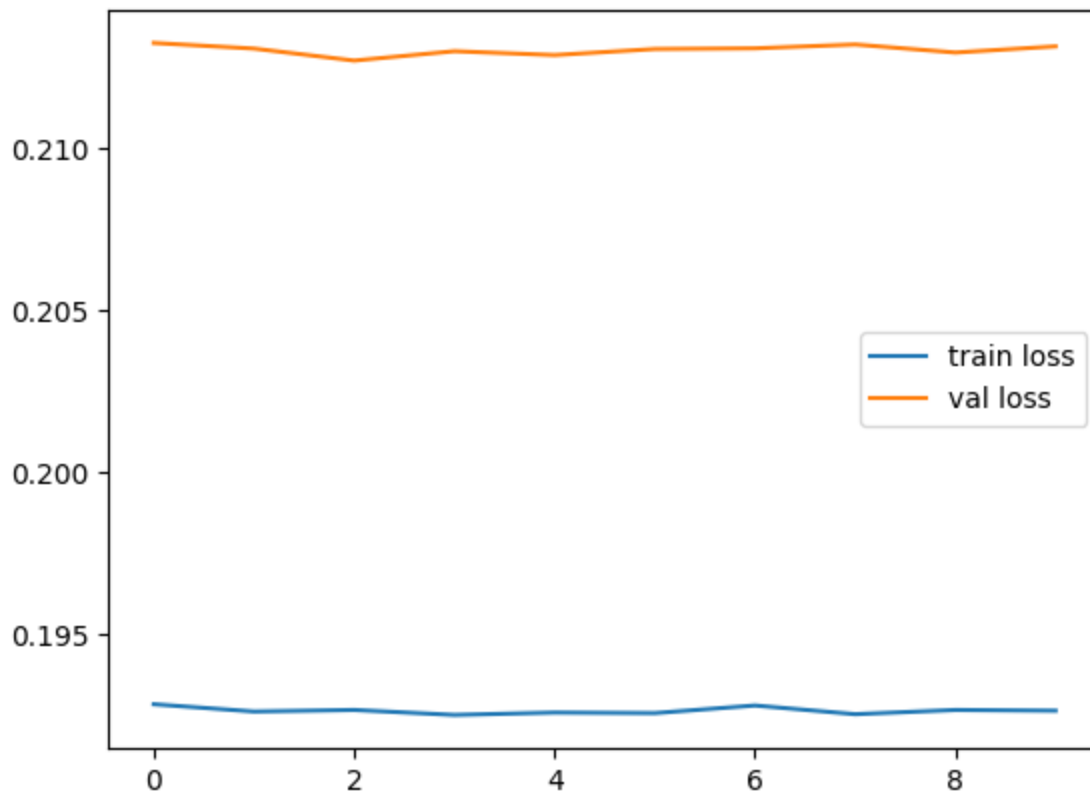
import nlpaug.augmenter.word as naw

# Now it will not crash
aug = naw.SynonymAug(aug_src="wordnet")
text = "The movie was fantastic and thrilling!"
print("Augmented:", aug.augment(text))

cnn_model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_len),
    tf.keras.layers.Conv1D(128, 5, activation="relu"),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.Dropout(0.5), # Dropout
    tf.keras.layers.Dense(64, activation="relu", kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

optimizer = Adam(learning_rate=1e-4)
cnn_model.compile(optimizer=optimizer, loss="binary_crossentropy", metrics=["accuracy"])

history = cnn_model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=10,
    batch_size=64,
    callbacks=[early_stop, lr_scheduler]
)
```



Augmented: ['The motion picture was fantastic and thrilling!']